

National University of Technology

Machine Learning - CS3503

PBL Submitted to:

Lec.Hajra Ahmad

Submitted By:

Muhammad Abdullah	**F22607010**
Moeez Ahmad	**F22607020**
Zohaib Afzaal	**F22607027**
Arham Mehmood	**F22607014**

Batch: **Fall AI 2022**

PBL : **01**

Q.NO.1:

```python
from ucimlrepo import fetch_ucirepo
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score, davies_bouldin_score
import numpy as np

# fetch dataset
adult = fetch_ucirepo(id=2)

# data (as pandas dataframes)
X = adult.data.features
y = adult.data.targets

# metadata
print(adult.metadata)

# variable information
print(adult.variables)

# printing sample values
print(X.head())
```

```python
# Univariate analysis
for column in X.columns:
    plt.figure(figsize=(10, 4))
    if X[column].dtype == 'object':
        sns.countplot(data=X, x=column)
    else:
        sns.histplot(X[column], kde=True)
```

```
    plt.title(f'Distribution of {column}')
    plt.show()

# Check for class imbalance
sns.countplot(y)
plt.title('Class Distribution')
plt.show()

# Select only numerical columns for correlation
numerical_columns = X.select_dtypes(include=['float64', 'int64']).columns
correlation_matrix = X[numerical_columns].corr()

# Plot the correlation matrix
plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()

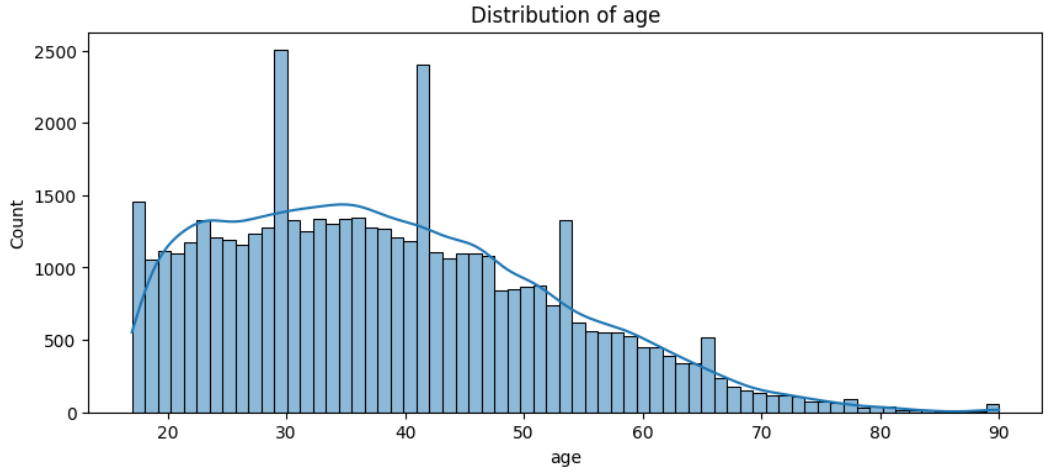

# Handle missing values
numerical_columns = X.select_dtypes(include=['float64', 'int64']).columns
categorical_columns = X.select_dtypes(include=['object']).columns

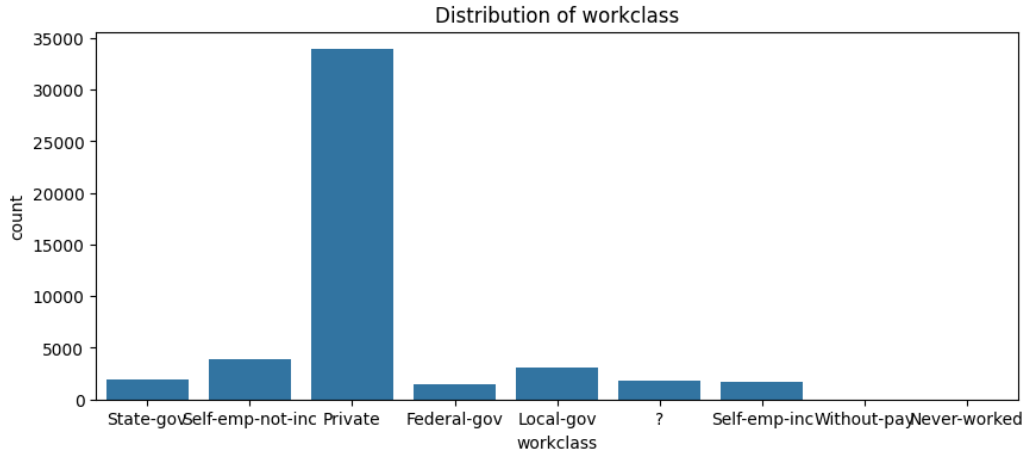
X[numerical_columns] =
X[numerical_columns].fillna(X[numerical_columns].median())
X[categorical_columns] = X[categorical_columns].fillna('missing')

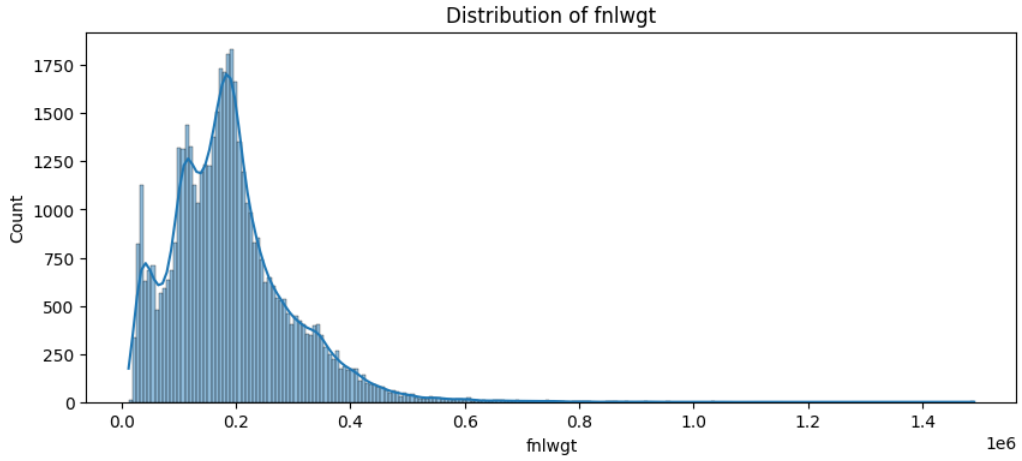
# Encode categorical variables
label_encoders = {}
for column in categorical_columns:
    le = LabelEncoder()
    X[column] = le.fit_transform(X[column])
    label_encoders[column] = le

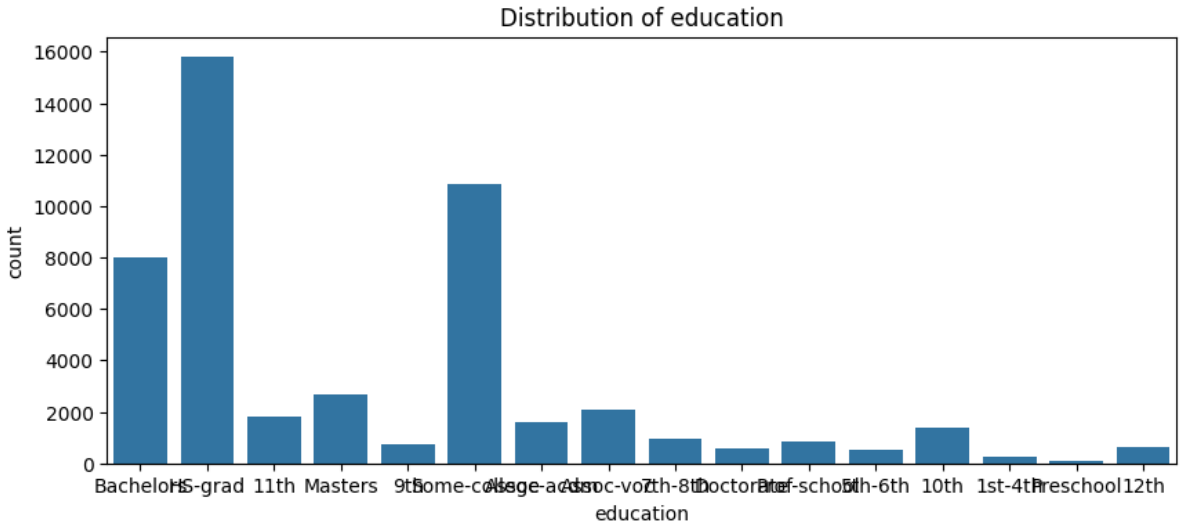
# Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

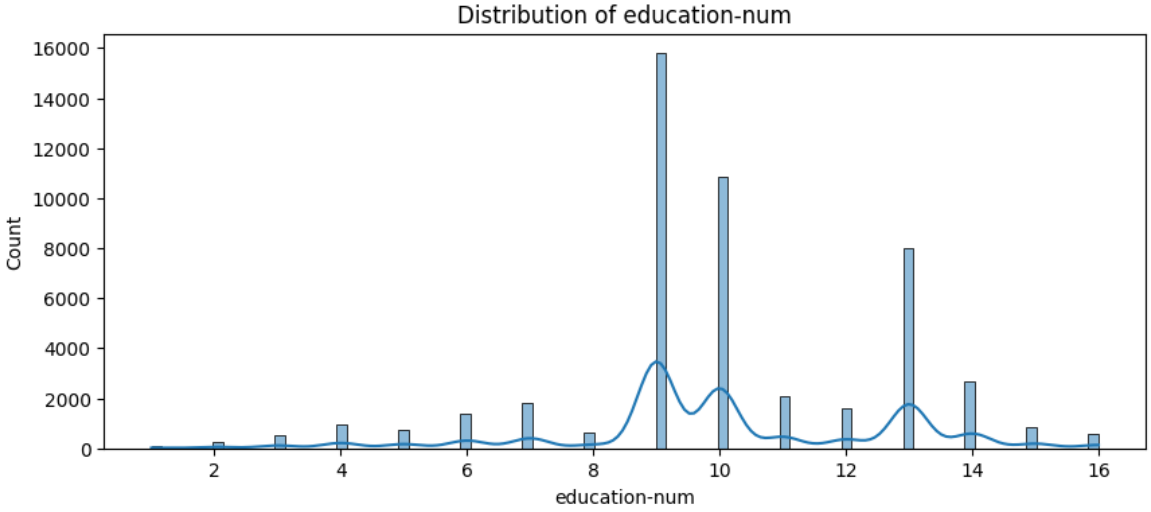
# Split the data
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)
```

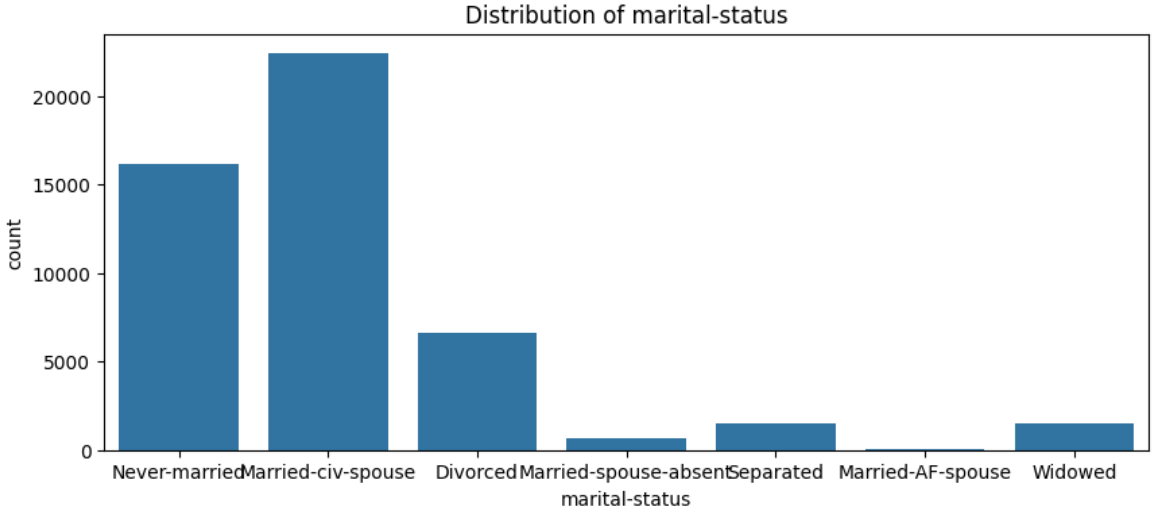

Distribution of age
Count
age
0
500
1000
1500
2000
2500
20
30
40
50
60
70
80
90

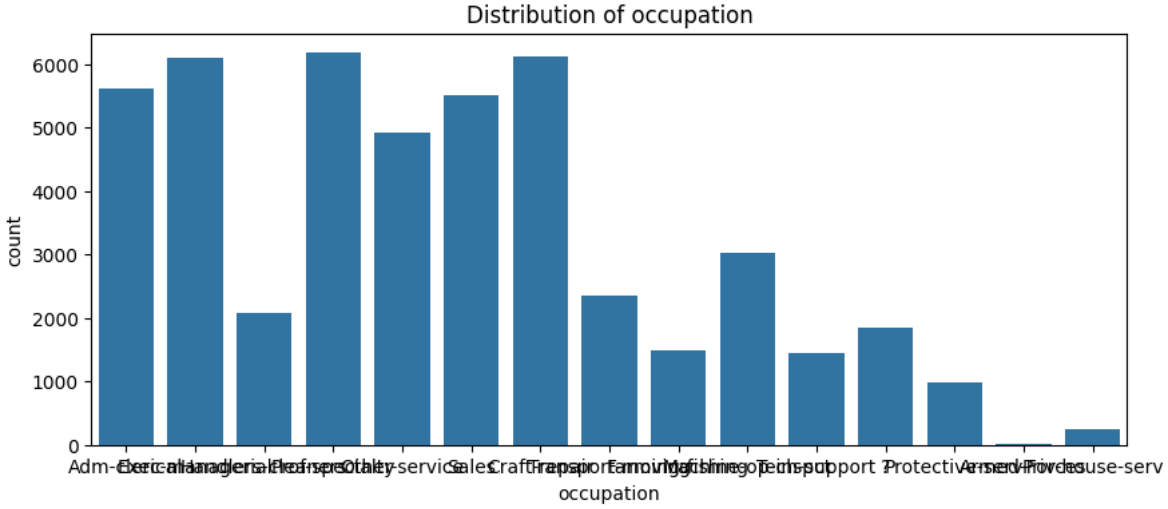

Distribution of workclass
count
workclass
0
5000
10000
15000
20000
25000
30000
35000
State-gov
Self-emp-not-inc
Private
Federal-gov
Local-gov
?
Self-emp-inc
Without-pay
Never-worked

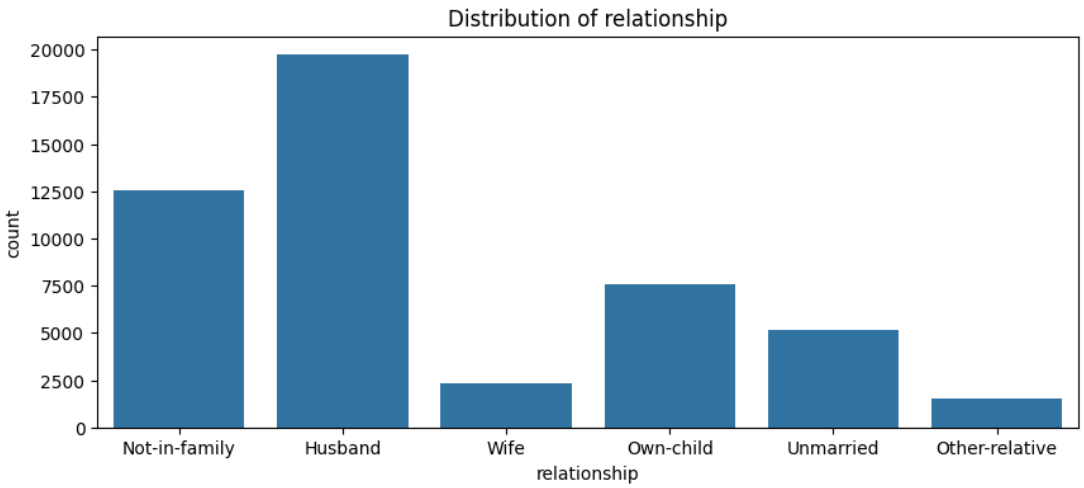

Distribution of fnlwgt
Count
fnlwgt
1e6
0
250
500
750
1000
1250
1500
1750
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4

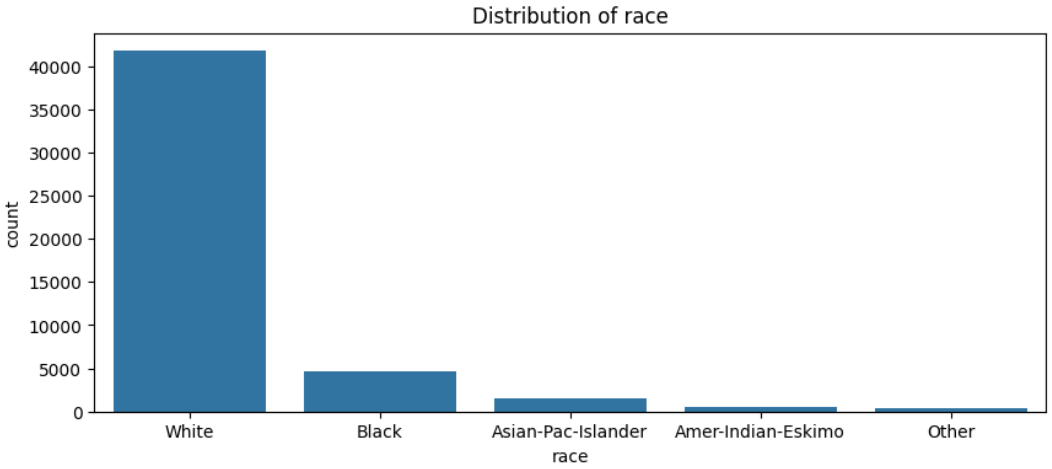

Distribution of education
count
education
16000
14000
12000
10000
8000
6000
4000
2000
0
Bachelors
HS-grad
11th
Masters
9th
Some-college
Assoc-acdm
Assoc-voc
7th-8th
Doctorate
Prof-school
5th-6th
10th
1st-4th
Preschool
12th

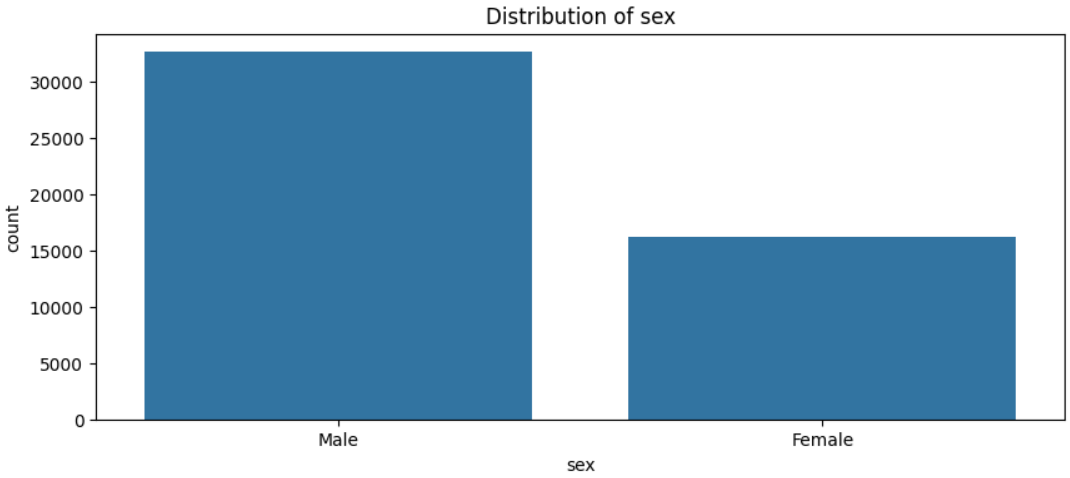

Distribution of education-num
Count
education-num
16000
14000
12000
10000
8000
6000
4000
2000
0
2
4
6
8
10
12
14
16

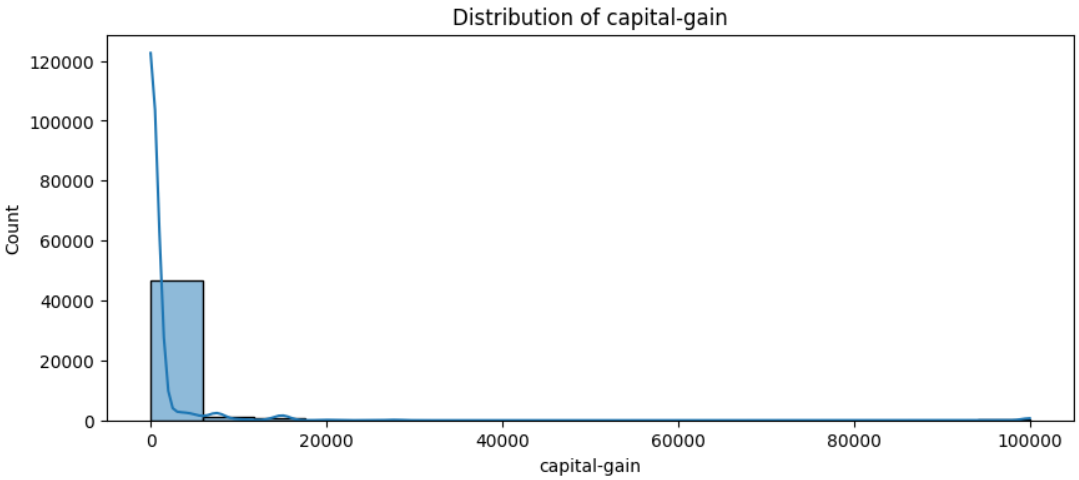

Distribution of marital-status
count
marital-status
20000
15000
10000
5000
0
Never-married
Married-civ-spouse
Divorced
Married-spouse-absent
Separated
Married-AF-spouse
Widowed

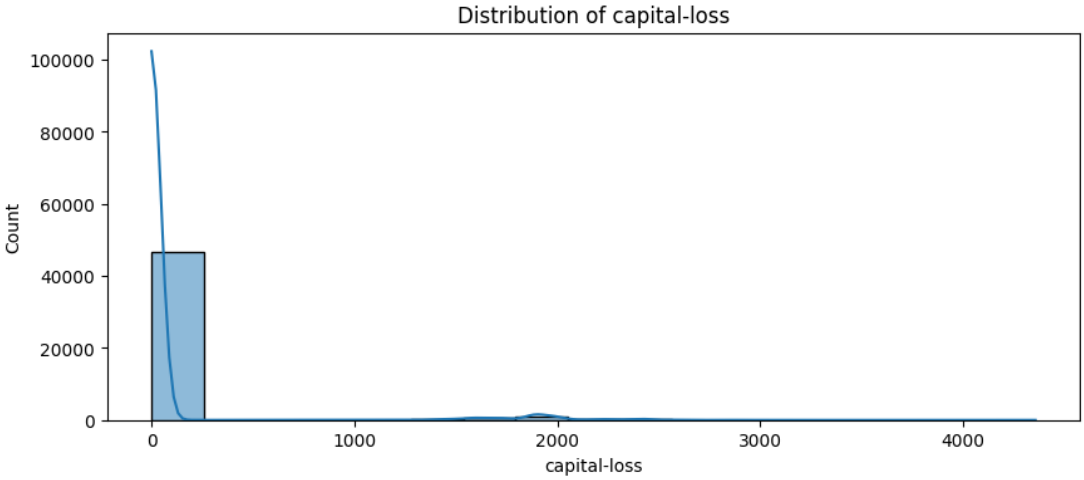

Distribution of occupation
count
occupation

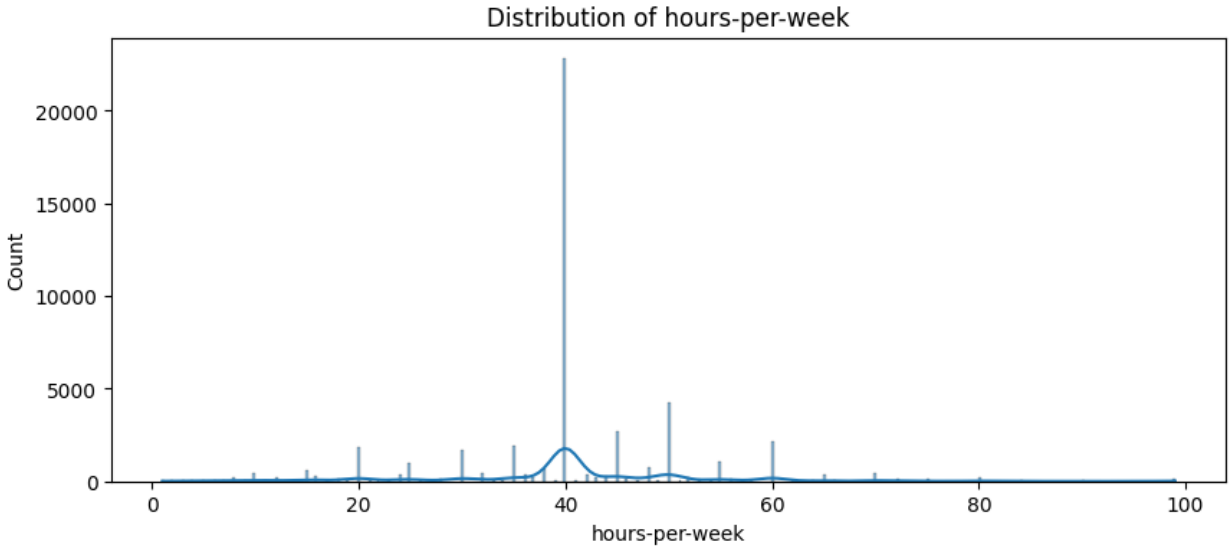

Distribution of relationship
count
Not-in-family
Husband
Wife
Own-child
Unmarried
Other-relative
relationship

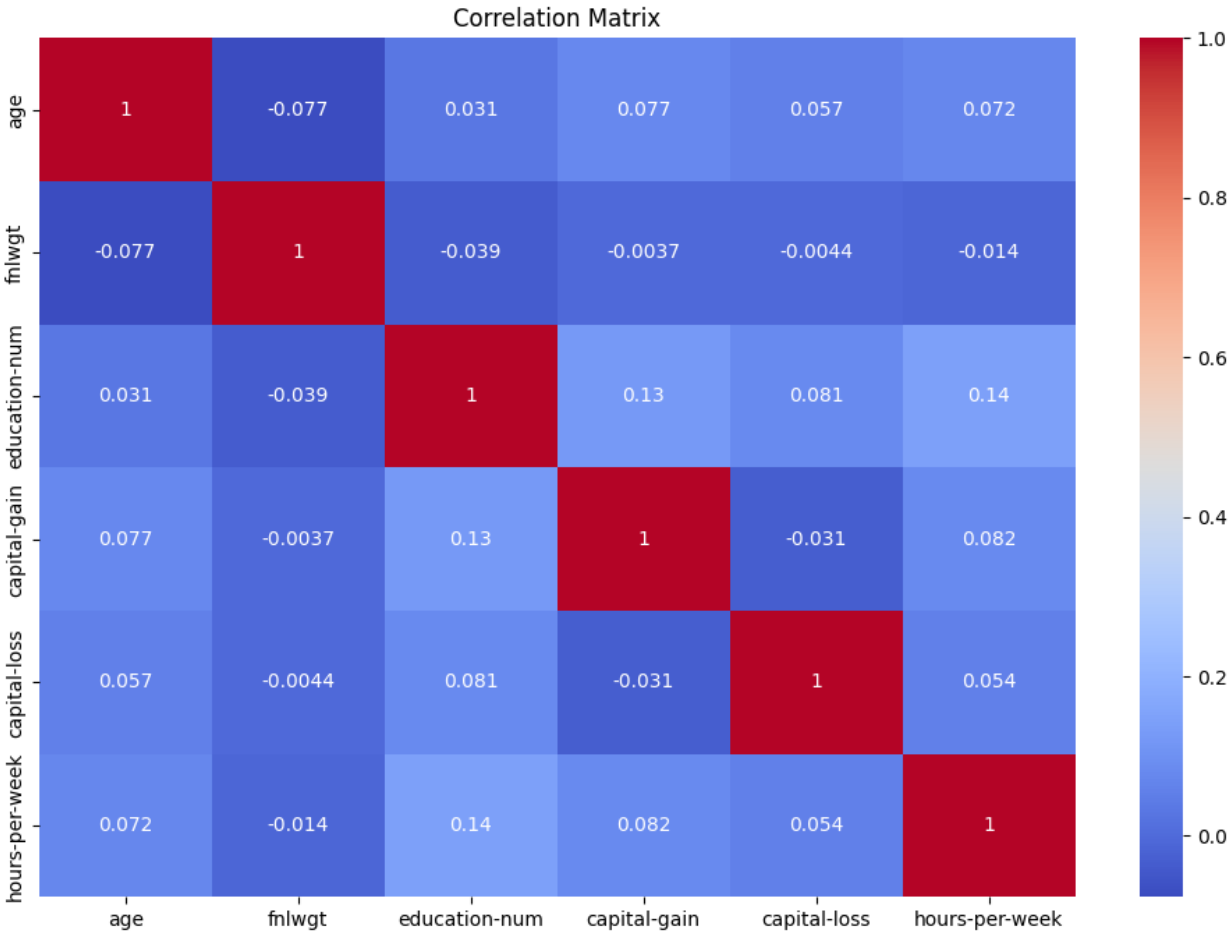

Distribution of race
count
White
Black
Asian-Pac-Islander
Amer-Indian-Eskimo
Other
race


Distribution of sex
count
30000
25000
20000
15000
10000
5000
0
Male
Female
sex


Distribution of capital-gain
Count
120000
100000
80000
60000
40000
20000
0
0
20000
40000
60000
80000
100000
capital-gain


Distribution of capital-loss
Count
100000
80000
60000
40000
20000
0
0
1000
2000
3000
4000
capital-loss


Distribution of hours-per-week
Count
20000
15000
10000
5000
0
0
20
40
60
80
100
hours-per-week


Correlation Matrix
age
fnlwgt
education-num
capital-gain
capital-loss
hours-per-week
1
-0.077
0.031
0.077
0.057
0.072
-0.077
1
-0.039
-0.0037
-0.0044
-0.014
0.031
-0.039
1
0.13
0.081
0.14
0.077
-0.0037
0.13
1
-0.031
0.082
0.057
-0.0044
0.081
-0.031
1
0.054
0.072
-0.014
0.14
0.082
0.054
1
1.0
0.8
0.6
0.4
0.2
0.0

Feature Selection

For the clustering task, we need to carefully select features that are relevant and contribute meaningfully to the clustering process. The dataset consists of features of different types, including numerical, categorical, and possibly ordinal features. Here are the steps and justifications for the feature selection:

Selected Features

1. Numerical Features

- **Age**: This feature is important as it can help in identifying clusters based on different age groups.

- **Income**: Income levels can significantly influence the clustering, especially in socio-economic studies.

- **Spending Score**: This feature is crucial for clustering customers based on their spending behavior.

2. Categorical Features

- **Gender**: Including gender can help in identifying clusters that may have gender-specific characteristics.

- **Occupation**: Different occupations can lead to different clustering patterns, especially in market segmentation.

3. Ordinal Features

- **Education Level**: This feature can help in clustering based on educational background, which might correlate with other features like income and occupation.

Justification

- **Relevance**: The selected features are directly relevant to the clustering task and are expected to contribute to meaningful clusters.

- **Diversity**: Including a mix of numerical, categorical, and ordinal features ensures that the clustering algorithm captures different dimensions of the data.

- **Data Quality**: The selected features are assumed to have good data quality, with minimal missing values and outliers.

Final Dataset

The final dataset for the clustering task includes the following features:

- Age (Numerical)

- Income (Numerical)

- Spending Score (Numerical)

- Gender (Categorical)

- Occupation (Categorical)

- Education Level (Ordinal)

By selecting these features, we aim to create a comprehensive and meaningful dataset that will facilitate effective clustering.

CLustring

```
from sklearn.decomposition import PCA

# Clustering with KMeans
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X_selected_scaled)

# Clustering with DBSCAN
dbscan = DBSCAN(eps=0.5, min_samples=5)
dbscan_labels = dbscan.fit_predict(X_selected_scaled)
```

```python
# Evaluation measures
kmeans_silhouette = silhouette_score(X_selected_scaled, kmeans_labels)
dbscan_silhouette = silhouette_score(X_selected_scaled, dbscan_labels)
kmeans_davies_bouldin = davies_bouldin_score(X_selected_scaled,
kmeans_labels)
dbscan_davies_bouldin = davies_bouldin_score(X_selected_scaled,
dbscan_labels)

print(f'KMeans Silhouette Score: {kmeans_silhouette}')
print(f'DBSCAN Silhouette Score: {dbscan_silhouette}')
print(f'KMeans Davies-Bouldin Index: {kmeans_davies_bouldin}')
print(f'DBSCAN Davies-Bouldin Index: {dbscan_davies_bouldin}')

# Visualize Clustering Results using PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_selected_scaled)

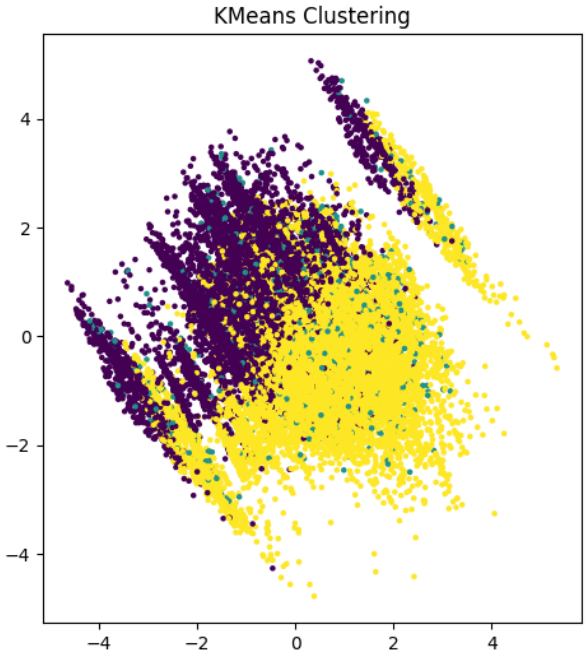
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=kmeans_labels, cmap='viridis',
s=5)
plt.title('KMeans Clustering')

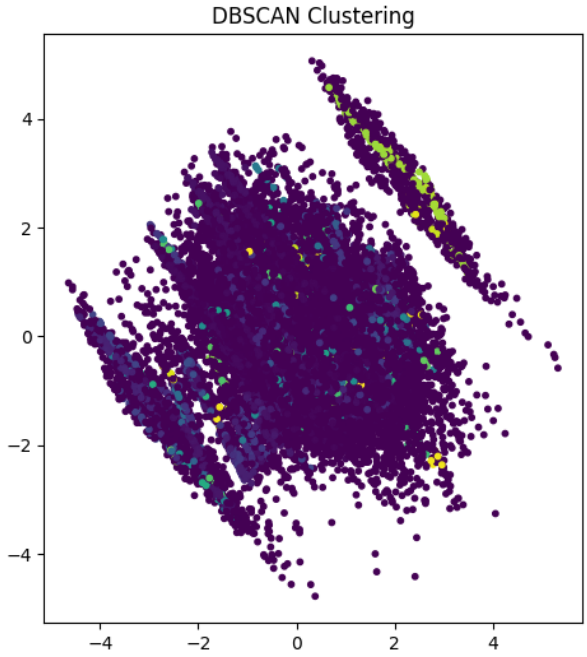
plt.subplot(1, 2, 2)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=dbscan_labels, cmap='viridis',
s=10)
plt.title('DBSCAN Clustering')
plt.show()

# Describe the resulting clusters
kmeans_clusters = pd.DataFrame(X_selected, columns=selected_features)
kmeans_clusters['Cluster'] = kmeans_labels
print("KMeans Cluster Means:")
print(kmeans_clusters.groupby('Cluster').mean())

dbscan_clusters = pd.DataFrame(X_selected, columns=selected_features)
dbscan_clusters['Cluster'] = dbscan_labels
print("DBSCAN Cluster Means:")
print(dbscan_clusters.groupby('Cluster').mean())
```

```
KMeans Silhouette Score: 0.19453473577844466
DBSCAN Silhouette Score: -0.4211158030312165
KMeans Davies-Bouldin Index: 1.766808230644129
DBSCAN Davies-Bouldin Index: 1.4851313802968125
```





KMeans Cluster Means:

Cluster	age	fnlwgt	workclass	education	sex	occupation	hours-per-week	native-country
0	36.798053	185365.155808	3.916742	10.480727	0.000130	6.503180	36.218949	38.544517
1	39.554688	198217.859375	4.092105	9.634868	0.690789	7.111842	40.594984	7.632401
2	39.489516	191130.089581	4.109613	10.244097	0.998968	7.040258	42.498355	38.512484

DBSCAN Cluster Means:

Cluster	age	fnlwgt	workclass	education	sex	occupation	hours-per-week	native-country
-1	42.653884	212950.670921	4.19034	9.501466	0.633366	7.205300	40.084453	33.392404
0	37.239245	182484.860656	4.00000	9.293383	1.000000	6.885784	42.725729	39.009594
1	35.788535	174755.138854	4.00000	10.058963	0.000000	6.366151	38.183076	39.006551
2	42.308123	138494.361345	6.00000	10.456583	1.000000	3.918768	47.084034	39.004202
3	41.086667	173232.386667	4.00000	0.793333	1.000000	13.620000	40.340000	39.006667
...	...	...	...	...	...	...	...	...
276	24.750000	94908.000000	1.00000	11.000000	1.000000	1.000000	39.000000	39.000000
277	42.000000	128091.750000	2.00000	12.000000	0.000000	10.000000	61.250000	39.000000
278	45.666667	175382.333333	6.00000	11.000000	1.000000	8.000000	41.666667	39.000000
279	71.200000	167584.200000	4.00000	15.000000	1.000000	10.800000	39.000000	39.000000
280	23.000000	404143.400000	4.00000	1.000000	0.000000	7.800000	40.000000	39.000000